BASIC SQL
ASSIGNMENT 2
HEALTHCARE MANAGEMENT SYSTEM

SUBMITTED BY: HARSHPREET KAUR
(harshpreet.kaur@kenexai.com)

TASK 1

ENTITIES

Entity	Primary Key
Patients	patient_id
Doctors	doctor_id
Departments	dept_id
Appointments	appt_id
Medical_Records	record_id
Prescriptions	prescription_id
Billings	bill_id

RELATIONSHIPS AND CARDINALITIES

-One-to-Many: Department↔Doctors

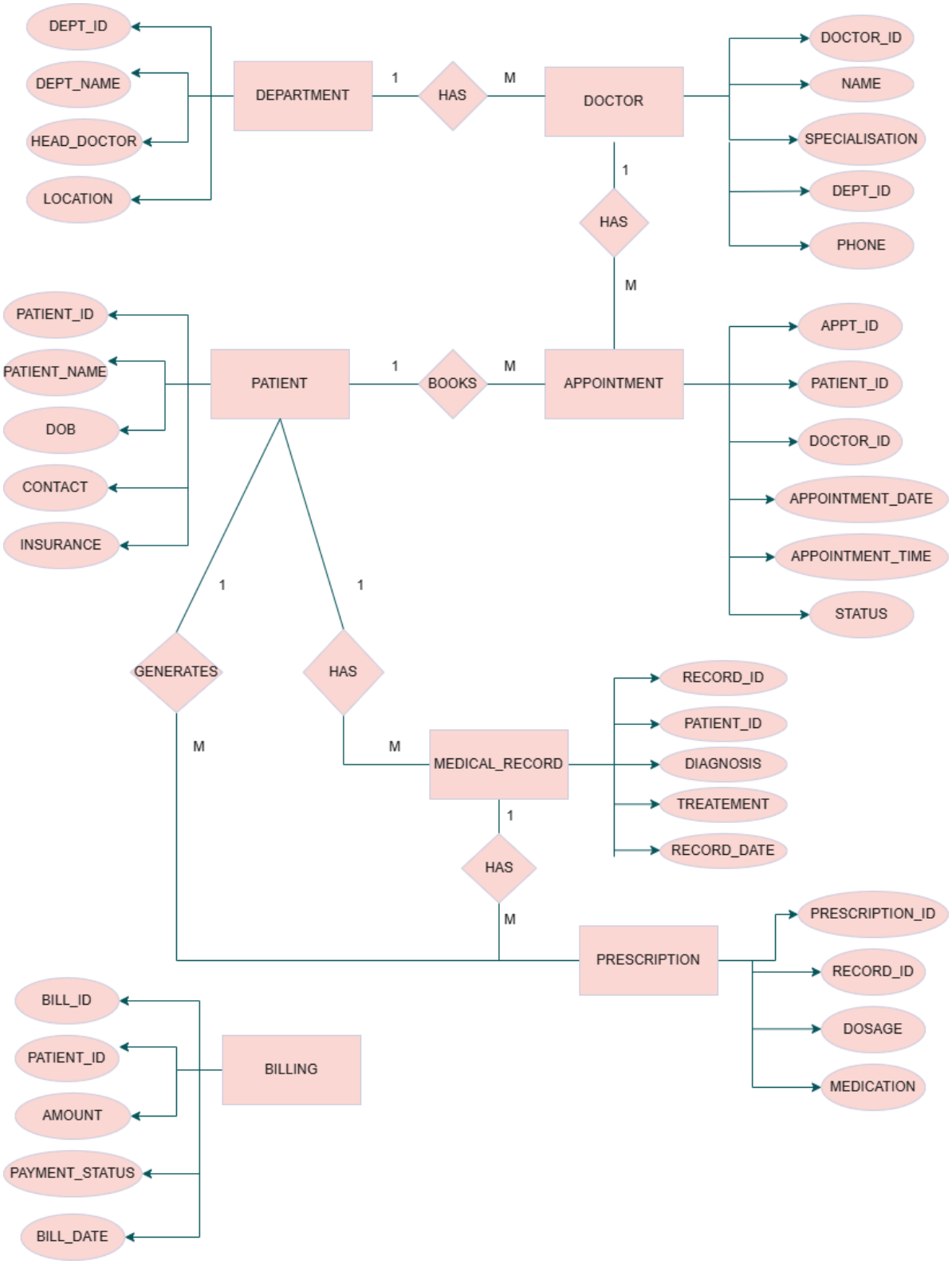
- Many-to-Many: Patients↔Doctors (through Appointments)

- One-to-Many: Patients↔Medical Records

- One-to-Many: Medical Records↔Prescriptions

Relationship	Cardinality
Department → Doctor	1:M
Patient → Appointment	1:M
Doctor → Appointment	1:M
Patient ↔ Doctor	M:N (through Appointment)
Patient → Medical_Record	1:M
Medical_Record → Prescription	1:M

FINAL ER DIAGRAM



TASK 2 : DATABASE AND TABLE CREATION

PART A:

```
CREATE DATABASE HealthCarePlus_DB;
GO
USE HealthCarePlus_DB;
GO
```

```
Commands completed successfully.

Completion time: 2026-06-10T12:45:13.6735267+05:30
```

PART B:

```
CREATE TABLE Department
(
   dept_id INT IDENTITY(1,1) PRIMARY KEY,

   dept_name VARCHAR(100) NOT NULL,

   location VARCHAR(100) NOT NULL,

   head_doctor_id INT NULL
);

CREATE TABLE Patient
(
   patient_id INT IDENTITY(1,1) PRIMARY KEY,

   first_name VARCHAR(50) NOT NULL,

   last_name VARCHAR(50) NOT NULL,

   date_of_birth DATE NOT NULL,

   gender VARCHAR(10)
      CHECK (gender IN ('Male','Female','Other')),

   phone VARCHAR(15) NOT NULL,

   email VARCHAR(100) UNIQUE,

   address VARCHAR(255),

   blood_group VARCHAR(5)
      CHECK
      (
         blood_group IN
         ('A+','A-','B+','B-','AB+','AB-','O+','O-')
      ),
```

```
    insurance_provider VARCHAR(100),

    registration_date DATE
        DEFAULT GETDATE(),

    is_active BIT
        DEFAULT 1
);

CREATE TABLE Doctor
(
    doctor_id INT IDENTITY(1,1) PRIMARY KEY,

    first_name VARCHAR(50) NOT NULL,

    last_name VARCHAR(50) NOT NULL,

    specialization VARCHAR(100) NOT NULL,

    phone VARCHAR(15) NOT NULL,

    email VARCHAR(100) UNIQUE,

    dept_id INT NOT NULL,

    FOREIGN KEY (dept_id)
    REFERENCES Department(dept_id)
);

CREATE TABLE Appointment
(
    appt_id INT IDENTITY(1,1) PRIMARY KEY,

    patient_id INT NOT NULL,

    doctor_id INT NOT NULL,

    appointment_date DATE NOT NULL,

    appointment_time TIME NOT NULL,

    status VARCHAR(20)
        DEFAULT 'Scheduled'

        CHECK
        (
            status IN
            (
                'Scheduled',
                'Completed',
                'Cancelled'
            )
        ),
```

```
    FOREIGN KEY (patient_id)
    REFERENCES Patient(patient_id),

    FOREIGN KEY (doctor_id)
    REFERENCES Doctor(doctor_id)
);

CREATE TABLE Medical_Record
(
    record_id INT IDENTITY(1,1) PRIMARY KEY,

    patient_id INT NOT NULL,

    diagnosis VARCHAR(255) NOT NULL,

    treatment VARCHAR(255),

    record_date DATE
        DEFAULT GETDATE(),

    FOREIGN KEY (patient_id)
    REFERENCES Patient(patient_id)
);

CREATE TABLE Prescription
(
    prescription_id INT IDENTITY(1,1) PRIMARY KEY,

    record_id INT NOT NULL,

    medication VARCHAR(100) NOT NULL,

    dosage VARCHAR(100) NOT NULL,

    FOREIGN KEY (record_id)
    REFERENCES Medical_Record(record_id)
);

CREATE TABLE Billing
(
    bill_id INT IDENTITY(1,1) PRIMARY KEY,

    patient_id INT NOT NULL,

    amount DECIMAL(10,2)
        CHECK (amount >= 0),

    payment_status VARCHAR(20)
        DEFAULT 'Pending'

        CHECK
        (
            payment_status IN
```

```
        (
            'Pending',
            'Paid',
            'Cancelled'
        )
    ),

    bill_date DATE
        DEFAULT GETDATE(),

    FOREIGN KEY (patient_id)
    REFERENCES Patient(patient_id)
);

-- add head_doctor foreign key

ALTER TABLE Department
ADD CONSTRAINT FK_Department_HeadDoctor
FOREIGN KEY (head_doctor_id)
REFERENCES Doctor(doctor_id);

--verify
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE';
```

Results | Messages

	TABLE_NAME
1	Department
2	Patient
3	Doctor
4	Appointment
5	Medical_Record
6	Prescription
7	Billing

TASK 3: DATA TYPE SELECTION

Category	Data Type	Example Field(s)	Reason for Selection
Numeric	**INT**	patient_id, doctor_id, dept_id, appt_id, record_id, prescription_id, bill_id	Efficient storage and indexing for primary and foreign keys. Supports `IDENTITY(1,1)` for auto-incrementing IDs in SQL Server.
Numeric	**DECIMAL(10,2)**	amount	Stores billing amounts with exact precision and two decimal places, making it suitable for financial data.
Numeric	**BIT**	is_active	Efficient storage for binary status flags (0 = inactive, 1 = active).
String	**VARCHAR(n)**	first_name, last_name, email, specialization, medication, insurance_provider	Variable-length storage optimizes space usage while supporting varying input lengths.
String	**CHAR(n)**	gender, blood_group	Suitable for fixed-length values and predefined codes.
String	**VARCHAR(MAX)**	diagnosis, treatment	Supports long medical descriptions and treatment notes that may exceed normal VARCHAR limits.
Date/Time	**DATE**	date_of_birth, registration_date, bill_date, record_date	Stores only the date component without time, reducing storage and improving clarity.
Date/Time	**DATETIME**	appointment_date_time	Stores both date and time information for appointment scheduling and event tracking.

Other (Version Tracking)	**ROWVERSION**	created_at, updated_at tracking	SQL Server's equivalent of TIMESTAMP. Automatically changes whenever a row is modified and helps with concurrency control.
Other (Restricted Values)	**VARCHAR + CHECK Constraint**	appointment_status, payment_status	SQL Server does not support ENUM. A CHECK constraint restricts values to predefined options such as 'Scheduled', 'Completed', and 'Cancelled'.

TASK 4: DATA INSERTION

```
--insert departments
INSERT INTO Department
(dept_name, location, head_doctor_id)
VALUES
('Cardiology','Ahmedabad',NULL),
('Neurology','Mumbai',NULL),
('Pediatrics','Delhi',NULL),
('Orthopedics','Pune',NULL),
('Emergency','Bangalore',NULL);

--insert doctors

INSERT INTO Doctor
(
first_name,
last_name,
specialization,
phone,
email,
dept_id
)
VALUES
('Amit','Sharma','Cardiologist','+91-9876543210','amit.sharma@email.com',1),
('Neha','Patel','Cardiologist','+91-9876543211','neha.patel@email.com',1),

('Raj','Verma','Neurologist','+91-9876543212','raj.verma@email.com',2),
('Priya','Singh','Neurologist','+91-9876543213','priya.singh@email.com',2),

('Anjali','Gupta','Pediatrician','+91-9876543214','anjali.gupta@email.com',3),
('Karan','Mehta','Pediatrician','+91-9876543215','karan.mehta@email.com',3),

('Vikas','Joshi','Orthopedic Surgeon','+91-9876543216','vikas.joshi@email.com',4),
('Ritu','Nair','Orthopedic Surgeon','+91-9876543217','ritu.nair@email.com',4),

('Suresh','Iyer','Emergency Specialist','+91-9876543218','suresh.iyer@email.com',5),
('Pooja','Reddy','Emergency Specialist','+91-9876543219','pooja.reddy@email.com',5);

-- assign department heads

UPDATE Department
SET head_doctor_id =
CASE dept_id
WHEN 1 THEN 1
WHEN 2 THEN 3
WHEN 3 THEN 5
WHEN 4 THEN 7
WHEN 5 THEN 9
END;

-- insert 20 patients

INSERT INTO Patient
```

```
(
first_name,last_name,date_of_birth,gender,
phone,email,address,blood_group,
insurance_provider
)
VALUES
('Rahul','Shah','1990-05-10','Male','+91-9000000001','rahul.shah@email.com','Ahmedabad','A+','Star Health'),
('Sneha','Patel','1988-08-20','Female','+91-9000000002','sneha.patel@email.com','Mumbai','B+','ICICI Lombard'),
('Arjun','Mehta','1995-03-15','Male','+91-9000000003','arjun.mehta@email.com','Delhi','O+','HDFC Ergo'),
('Kavya','Rao','1992-11-22','Female','+91-9000000004','kavya.rao@email.com','Pune','AB+','Star Health'),
('Rohan','Verma','1985-07-05','Male','+91-9000000005','rohan.verma@email.com','Bangalore','A-','Bajaj Allianz'),
('Priya','Nair','1998-09-14','Female','+91-9000000006','priya.nair@email.com','Chennai','O-','Niva Bupa'),
('Aditya','Gupta','1991-02-18','Male','+91-9000000007','aditya.gupta@email.com','Jaipur','B-','Care Health'),
('Pallavi','Joshi','1994-12-01','Female','+91-9000000008','pallavi.joshi@email.com','Surat','AB-','ICICI Lombard'),
('Nitin','Singh','1987-06-12','Male','+91-9000000009','nitin.singh@email.com','Lucknow','A+','Star Health'),
('Meera','Kapoor','1993-01-30','Female','+91-9000000010','meera.kapoor@email.com','Indore','B+','HDFC Ergo'),

('Aakash','Sharma','1997-04-11','Male','+91-9000000011','aakash.sharma@email.com','Ahmedabad','O+','Care
Health'),
('Ritika','Das','1996-10-09','Female','+91-9000000012','ritika.das@email.com','Kolkata','AB+','Star Health'),
('Vivek','Reddy','1989-08-03','Male','+91-9000000013','vivek.reddy@email.com','Hyderabad','A-','Niva Bupa'),
('Ananya','Iyer','1991-05-17','Female','+91-9000000014','ananya.iyer@email.com','Chennai','O-','ICICI Lombard'),
('Deepak','Malhotra','1984-09-21','Male','+91-9000000015','deepak.m@email.com','Delhi','B+','Bajaj Allianz'),
('Pooja','Kulkarni','1999-02-13','Female','+91-9000000016','pooja.k@email.com','Pune','A+','Star Health'),
('Sanjay','Tiwari','1986-03-28','Male','+91-9000000017','sanjay.t@email.com','Kanpur','O+','Care Health'),
('Neelam','Bansal','1990-07-19','Female','+91-9000000018','neelam.b@email.com','Jaipur','AB-','HDFC Ergo'),
('Harsh','Chopra','1995-12-07','Male','+91-9000000019','harsh.c@email.com','Mumbai','A+','Niva Bupa'),
('Divya','Menon','1993-06-25','Female','+91-9000000020','divya.m@email.com','Kochi','B-','Star Health');

SELECT COUNT(*) AS Departments FROM Department;
SELECT COUNT(*) AS Doctors FROM Doctor;
SELECT COUNT(*) AS Patients FROM Patient;
```

	Departments
1	5

	Doctors
1	10

	Patients
1	20

```
--insert 30 appointments

INSERT INTO Appointment
(patient_id, doctor_id, appointment_date, appointment_time, status)
VALUES
(1,1,'2025-01-10','09:00','Completed'),
(2,2,'2025-01-12','10:00','Completed'),
(3,3,'2025-01-15','11:00','Completed'),
(4,4,'2025-01-18','14:00','Completed'),
```

```
(5,5,'2025-01-20','15:00','Completed'),
(6,6,'2025-01-22','09:30','Completed'),
(7,7,'2025-01-25','10:30','Completed'),
(8,8,'2025-01-28','11:30','Completed'),
(9,9,'2025-02-01','12:00','Completed'),
(10,10,'2025-02-03','13:00','Completed'),

(11,1,'2025-02-05','09:00','Completed'),
(12,2,'2025-02-08','10:00','Completed'),
(13,3,'2025-02-10','11:00','Completed'),
(14,4,'2025-02-12','14:00','Completed'),
(15,5,'2025-02-15','15:00','Completed'),

(16,6,'2026-07-10','09:00','Scheduled'),
(17,7,'2026-07-12','10:00','Scheduled'),
(18,8,'2026-07-15','11:00','Scheduled'),
(19,9,'2026-07-18','14:00','Scheduled'),
(20,10,'2026-07-20','15:00','Scheduled'),

(1,3,'2026-07-22','09:00','Scheduled'),
(2,4,'2026-07-25','10:00','Scheduled'),
(3,5,'2026-07-28','11:00','Scheduled'),
(4,6,'2026-08-01','12:00','Scheduled'),
(5,7,'2026-08-03','13:00','Scheduled'),

(6,8,'2025-03-10','09:00','Cancelled'),
(7,9,'2025-03-12','10:00','Cancelled'),
(8,10,'2025-03-15','11:00','Cancelled'),
(9,1,'2025-03-18','14:00','Cancelled'),
(10,2,'2025-03-20','15:00','Cancelled');

--insert 15 medical records
INSERT INTO Medical_Record
(patient_id, diagnosis, treatment, record_date)
VALUES
(1,'Hypertension','Blood pressure medication','2025-01-10'),
(2,'Heart Arrhythmia','Cardiac monitoring','2025-01-12'),
(3,'Migraine','Pain management therapy','2025-01-15'),
(4,'Epilepsy','Medication prescribed','2025-01-18'),
(5,'Asthma','Inhaler treatment','2025-01-20'),
(6,'Viral Fever','Rest and medication','2025-01-22'),
(7,'Fractured Arm','Casting procedure','2025-01-25'),
(8,'Knee Pain','Physiotherapy','2025-01-28'),
(9,'Food Poisoning','IV fluids','2025-02-01'),
(10,'Chest Infection','Antibiotics','2025-02-03'),
(11,'Diabetes','Diet and medication','2025-02-05'),
(12,'Anxiety','Counseling','2025-02-08'),
(13,'Back Pain','Physical therapy','2025-02-10'),
(14,'Allergy','Antihistamines','2025-02-12'),
(15,'High Cholesterol','Lifestyle changes','2025-02-15');

-- insert 25 prescriptions
INSERT INTO Prescription
(record_id, medication, dosage)
```

```
VALUES
(1,'Amlodipine','5mg daily'),
(1,'Losartan','50mg daily'),

(2,'Metoprolol','25mg daily'),
(2,'Aspirin','75mg daily'),

(3,'Sumatriptan','50mg as needed'),

(4,'Levetiracetam','500mg twice daily'),
(4,'Vitamin B Complex','1 tablet daily'),

(5,'Salbutamol Inhaler','2 puffs daily'),

(6,'Paracetamol','500mg three times daily'),

(7,'Calcium Supplement','1 tablet daily'),
(7,'Pain Reliever','As required'),

(8,'Ibuprofen','400mg twice daily'),

(9,'ORS','After every loose stool'),
(9,'Antibiotic','Twice daily'),

(10,'Azithromycin','500mg daily'),

(11,'Metformin','500mg twice daily'),
(11,'Vitamin D','Weekly'),

(12,'Escitalopram','10mg daily'),

(13,'Muscle Relaxant','Twice daily'),
(13,'Pain Gel','Apply locally'),

(14,'Cetirizine','10mg daily'),

(15,'Atorvastatin','20mg daily'),

(5,'Montelukast','10mg nightly'),
(10,'Cough Syrup','10ml twice daily'),
(3,'Naproxen','250mg daily');

-- insert 20 billing records
INSERT INTO Billing
(patient_id, amount, payment_status, bill_date)
VALUES
(1,2500,'Paid','2025-01-10'),
(2,12000,'Paid','2025-01-12'),
(3,1800,'Paid','2025-01-15'),
(4,15000,'Paid','2025-01-18'),
(5,2200,'Paid','2025-01-20'),

(6,1200,'Pending','2025-01-22'),
(7,18000,'Paid','2025-01-25'),
```

```
(8,3500,'Paid','2025-01-28'),
(9,5000,'Pending','2025-02-01'),
(10,4200,'Paid','2025-02-03'),

(11,2500,'Paid','2025-02-05'),
(12,3000,'Cancelled','2025-02-08'),
(13,4500,'Paid','2025-02-10'),
(14,2000,'Pending','2025-02-12'),
(15,8000,'Paid','2025-02-15'),

(16,1500,'Pending','2025-03-01'),
(17,25000,'Paid','2025-03-05'),
(18,900,'Paid','2025-03-08'),
(19,50000,'Pending','2025-03-12'),
(20,7000,'Paid','2025-03-15');

SELECT COUNT(*) AS Departments FROM Department;
SELECT COUNT(*) AS Doctors FROM Doctor;
SELECT COUNT(*) AS Patients FROM Patient;
SELECT COUNT(*) AS Appointments FROM Appointment;
SELECT COUNT(*) AS MedicalRecords FROM Medical_Record;
SELECT COUNT(*) AS Prescriptions FROM Prescription;
SELECT COUNT(*) AS BillingRecords FROM Billing;
```

	Departments
1	5

	Doctors
1	10

	Patients
1	20

	Appointments
1	30

	MedicalRecords
1	15

	Prescriptions
1	25

	BillingRecords
1	20

TASK 5: BASIC SELECT QUERIES

```sql
-- 1. Display all patients' names and contact information
SELECT
   first_name,
   last_name,
   phone,
   email,
   address
FROM Patient;
```

	first_name	last_name	phone	email	address
1	Rahul	Shah	+91-9000000001	rahul.shah@email.com	Ahmedabad
2	Sneha	Patel	+91-9000000002	sneha.patel@email.com	Mumbai
3	Arjun	Mehta	+91-9000000003	arjun.mehta@email.com	Delhi
4	Kavya	Rao	+91-9000000004	kavya.rao@email.com	Pune
5	Rohan	Verma	+91-9000000005	rohan.verma@email.com	Bangalore
6	Priya	Nair	+91-9000000006	priya.nair@email.com	Chennai
7	Aditya	Gupta	+91-9000000007	aditya.gupta@email.com	Jaipur
8	Pallavi	Joshi	+91-9000000008	pallavi.joshi@email.com	Surat
9	Nitin	Singh	+91-9000000009	nitin.singh@email.com	Lucknow
10	Meera	Kapoor	+91-9000000010	meera.kapoor@email.c...	Indore
11	Aakash	Sharma	+91-9000000011	aakash.sharma@email...	Ahmedabad
12	Ritika	Das	+91-9000000012	ritika.das@email.com	Kolkata
13	Vivek	Reddy	+91-9000000013	vivek.reddy@email.com	Hyderabad
14	Ananya	Iyer	+91-9000000014	ananya.iyer@email.com	Chennai
15	Deepak	Malhotra	+91-9000000015	deepak.m@email.com	Delhi
16	Pooja	Kulkarni	+91-9000000016	pooja.k@email.com	Pune
17	Sanjay	Tiwari	+91-9000000017	sanjay.t@email.com	Kanpur
18	Neelam	Bansal	+91-9000000018	neelam.b@email.com	Jaipur
19	Harsh	Chopra	+91-9000000019	harsh.c@email.com	Mumbai
20	Divya	Menon	+91-9000000020	divya.m@email.com	Kochi

```sql
--2. List all Doctors in Cardiology department
SELECT
   d.first_name,
   d.last_name,
   d.specialization
FROM Doctor d
INNER JOIN Department dp
ON d.dept_id = dp.dept_id
WHERE dp.dept_name = 'Cardiology';
```

	first_name	last_name	specialization
1	Amit	Sharma	Cardiologist
2	Neha	Patel	Cardiologist

```sql
--4. Find all patients with 'O+' Blood group
SELECT
    patient_id,
    first_name,
    last_name,
    blood_group
FROM Patient
WHERE blood_group = 'O+';

--5. Display billing records with amount > ₹10,000 ordered by amount DESC
SELECT *
FROM Billing
WHERE amount > 10000
ORDER BY amount DESC;

--6. List top 5 most expensive bills
SELECT TOP 5 *
FROM Billing
ORDER BY amount DESC;

--7. Count total number of patients registered in 2024
SELECT COUNT(*) AS total_patients_2024
FROM Patient
WHERE YEAR(registration_date) = 2024;

--8. Show unique specializations of doctors
SELECT DISTINCT specialization
FROM Doctor;

--9. Find patients whose names start with 'A'
SELECT
    patient_id,
    first_name,
    last_name
FROM Patient
WHERE first_name LIKE 'A%';

--10. Display appointments between '2024-01-01' and '2024-12-31'
SELECT *
FROM Appointment
WHERE appointment_date
BETWEEN '2024-01-01'
AND '2024-12-31';
```

TASK 6: STRING FUNCTIONS

```
--1. CONCAT
SELECT
   patient_id,
   CONCAT(first_name, ' ', last_name) AS full_name
FROM Patient;
```

	patient_id	full_name
1	1	Rahul Shah
2	2	Sneha Patel
3	3	Arjun Mehta
4	4	Kavya Rao
5	5	Rohan Verma
6	6	Priya Nair
7	7	Aditya Gupta
8	8	Pallavi Joshi
9	9	Nitin Singh
10	10	Meera Kapoor
11	11	Aakash Sharma
12	12	Ritika Das
13	13	Vivek Reddy
14	14	Ananya Iyer
15	15	Deepak Malho...
16	16	Pooja Kulkarni
17	17	Sanjay Tiwari
18	18	Neelam Bansal
19	19	Harsh Chopra
20	20	Divya Menon

```
--2. UPPER
SELECT
   UPPER(CONCAT(first_name, ' ', last_name)) AS doctor_name
FROM Doctor;
```

	doctor_name
1	AMIT SHARMA
2	NEHA PATEL
3	RAJ VERMA
4	PRIYA SINGH
5	ANJALI GUPTA
6	KARAN MEHTA
7	VIKAS JOSHI
8	RITU NAIR
9	SURESH IYER
10	POOJA REDDY

```sql
--3. SUBSTRING
SELECT
   first_name,
   SUBSTRING(first_name, 1, 3) AS first_three_characters
FROM Patient;
```

	first_name	first_three_characters
1	Rahul	Rah
2	Sneha	Sne
3	Arjun	Arj
4	Kavya	Kav
5	Rohan	Roh
6	Priya	Pri
7	Aditya	Adi
8	Pallavi	Pal
9	Nitin	Nit
10	Meera	Mee
11	Aakash	Aak
12	Ritika	Rit
13	Vivek	Viv
14	Ananya	Ana
15	Deepak	Dee
16	Pooja	Poo
17	Sanjay	San
18	Neelam	Nee
19	Harsh	Har
20	Divya	Div

```sql
--4. LENGTH
SELECT
   patient_id,
   CONCAT(first_name, ' ', last_name) AS full_name
FROM Patient
WHERE LEN(CONCAT(first_name, ' ', last_name)) > 15;

--5. TRIM
SELECT
   email,
   LTRIM(RTRIM(email)) AS cleaned_email
FROM Patient;

--6. REPLACE
SELECT
   REPLACE(
      CONCAT('Dr. ', first_name, ' ', last_name),
      'Dr.',
      'Doctor'
   ) AS doctor_name
FROM Doctor;

--7. CONCAT_WS
```

```sql
SELECT
    patient_id,
    CONCAT_WS(', ',
            address,
            blood_group,
            insurance_provider) AS formatted_details
FROM Patient;

--8. Create Email Format
SELECT
    first_name,
    last_name,
    LOWER(
        CONCAT(
            first_name,
            '.',
            last_name,
            '@healthcareplus.com'
        )
    ) AS generated_email
FROM Doctor;
```

TASK 7: DATE AND TIME FUNCTIONS

```
--1. Age Calculation
SELECT
   patient_id,
   first_name,
   last_name,
   date_of_birth,
   DATEDIFF(YEAR, date_of_birth, GETDATE())
   - CASE
      WHEN DATEADD(YEAR,
                DATEDIFF(YEAR, date_of_birth, GETDATE()),
                date_of_birth) > GETDATE()
      THEN 1
      ELSE 0
     END AS age
FROM Patient;
```

	patient_id	first_name	last_name	date_of_birth	age
1	1	Rahul	Shah	1990-05-10	36
2	2	Sneha	Patel	1988-08-20	37
3	3	Arjun	Mehta	1995-03-15	31
4	4	Kavya	Rao	1992-11-22	33
5	5	Rohan	Verma	1985-07-05	40
6	6	Priya	Nair	1998-09-14	27
7	7	Aditya	Gupta	1991-02-18	35
8	8	Pallavi	Joshi	1994-12-01	31
9	9	Nitin	Singh	1987-06-12	38
10	10	Meera	Kapoor	1993-01-30	33
11	11	Aakash	Sharma	1997-04-11	29
12	12	Ritika	Das	1996-10-09	29
13	13	Vivek	Reddy	1989-08-03	36
14	14	Ananya	Iyer	1991-05-17	35
15	15	Deepak	Malhotra	1984-09-21	41
16	16	Pooja	Kulkarni	1999-02-13	27
17	17	Sanjay	Tiwari	1986-03-28	40
18	18	Neelam	Bansal	1990-07-19	35
19	19	Harsh	Chopra	1995-12-07	30
20	20	Divya	Menon	1993-06-25	32

```
-- 2. Show Appointment Schedule today
SELECT *
FROM Appointment
WHERE appointment_date = CAST(GETDATE() AS DATE);
--3. Date format
SELECT
   appt_id,
   FORMAT(appointment_date, 'dd-MM-yyyy') + ' ' +
   FORMAT(CAST(appointment_time AS DATETIME), 'hh:mm tt')
   AS formatted_appointment
FROM Appointment;
```

```sql
--4. Find patients registered more than 365 days ago
SELECT *
FROM Patient
WHERE DATEDIFF(DAY, registration_date, GETDATE()) > 365;
```

	patient_id	first_name	last_name	date_of_birth	gender	phone	email	address	blood_group	insurance_provider	registration_date	is_active

```sql
--5. Find appointments scheduled within next 7 days
SELECT *
FROM Appointment
WHERE appointment_date
BETWEEN CAST(GETDATE() AS DATE)
AND DATEADD(DAY, 7, CAST(GETDATE() AS DATE));

--6. List all appointments grouped by month
SELECT
    YEAR(appointment_date) AS appointment_year,
    MONTH(appointment_date) AS appointment_month,
    COUNT(*) AS total_appointments
FROM Appointment
GROUP BY
    YEAR(appointment_date),
    MONTH(appointment_date)
ORDER BY
    appointment_year,
    appointment_month;
```

	appointment_year	appointment_month	total_appointments
1	2025	1	8
2	2025	2	7
3	2025	3	5
4	2026	7	8
5	2026	8	2

```sql
--7. Show which day of the week each appointment falls on
SELECT
    appt_id,
    appointment_date,
    DATENAME(WEEKDAY, appointment_date) AS day_name
FROM Appointment;

--8. Find patients born in the 1990s
SELECT *
FROM Patient
WHERE YEAR(date_of_birth)
BETWEEN 1990 AND 1999;
```

```sql
--9. Calculate days since last appointment for each patient
SELECT
    patient_id,
    MAX(appointment_date) AS last_appointment_date,
    DATEDIFF(
        DAY,
        MAX(appointment_date),
        GETDATE()
    ) AS days_since_last_appointment
FROM Appointment
GROUP BY patient_id;

--10. Show billing records from current month
SELECT *
FROM Billing
WHERE MONTH(bill_date) = MONTH(GETDATE())
AND YEAR(bill_date) = YEAR(GETDATE());
```

TASK 8: NUMERIC AND AGGREGATE FUNCTIONS

```sql
--1. SUM
SELECT
   SUM(amount) AS total_revenue
FROM Billing;
```

	total_revenue
1	169800.00

```sql
--2. AVERAGE
SELECT
   patient_id,
   AVG(amount) AS average_bill_amount
FROM Billing
GROUP BY patient_id;
```

	patient_id	average_bill_amount
1	1	2500.000000
2	2	12000.000000
3	3	1800.000000
4	4	15000.000000
5	5	2200.000000
6	6	1200.000000
7	7	18000.000000
8	8	3500.000000
9	9	5000.000000
10	10	4200.000000
11	11	2500.000000
12	12	3000.000000
13	13	4500.000000
14	14	2000.000000
15	15	8000.000000
16	16	1500.000000
17	17	25000.000000
18	18	900.000000
19	19	50000.000000
20	20	7000.000000

```sql
--3. COUNT
SELECT
   doctor_id,
   COUNT(*) AS total_appointments
FROM Appointment
GROUP BY doctor_id;
```

	doctor_id	total_appointments
1	1	3
2	2	3
3	3	3
4	4	3
5	5	3
6	6	3
7	7	3
8	8	3
9	9	3
10	10	3

```sql
--4. MAX/MIN
SELECT
    MAX(amount) AS highest_bill,
    MIN(amount) AS lowest_bill
FROM Billing;

--5. ROUND
SELECT
    bill_id,
    amount,
    ROUND(amount, -2) AS rounded_amount
FROM Billing;

--6. CEILING/FLOOR
SELECT
    bill_id,
    amount,
    CEILING(amount) AS rounded_up,
    FLOOR(amount) AS rounded_down
FROM Billing;

--7. GROUP BY
SELECT
    d.dept_name,
    SUM(b.amount) AS total_billing
FROM Department d
INNER JOIN Doctor doc
    ON d.dept_id = doc.dept_id
INNER JOIN Appointment a
    ON doc.doctor_id = a.doctor_id
INNER JOIN Billing b
    ON a.patient_id = b.patient_id
GROUP BY d.dept_name;

--8. HAVING
SELECT
    d.dept_name,
    COUNT(doc.doctor_id) AS total_doctors
FROM Department d
```

```sql
INNER JOIN Doctor doc
    ON d.dept_id = doc.dept_id
GROUP BY d.dept_name
HAVING COUNT(doc.doctor_id) > 2;

--9. Calculate total prescriptions per medical record
SELECT
    record_id,
    COUNT(*) AS total_prescriptions
FROM Prescription
GROUP BY record_id;

--10. Find department with highest total billing
SELECT TOP 1
    d.dept_name,
    SUM(b.amount) AS total_billing
FROM Department d
INNER JOIN Doctor doc
    ON d.dept_id = doc.dept_id
INNER JOIN Appointment a
    ON doc.doctor_id = a.doctor_id
INNER JOIN Billing b
    ON a.patient_id = b.patient_id
GROUP BY d.dept_name
ORDER BY total_billing DESC;
```

TASK 9: JOIN OPERATIONS

```
--1. INNER JOIN
SELECT
   a.appt_id,
   p.first_name + ' ' + p.last_name AS patient_name,
   d.first_name + ' ' + d.last_name AS doctor_name,
   a.appointment_date,
   a.appointment_time,
   a.status
FROM Appointment a
INNER JOIN Patient p
   ON a.patient_id = p.patient_id
INNER JOIN Doctor d
   ON a.doctor_id = d.doctor_id;
```

	appt_id	patient_name	doctor_name	appointment_date	appointment_time	status
1	1	Rahul Shah	Amit Sharma	2025-01-10	09:00:00.0000000	Completed
2	2	Sneha Patel	Neha Patel	2025-01-12	10:00:00.0000000	Completed
3	3	Arjun Mehta	Raj Verma	2025-01-15	11:00:00.0000000	Completed
4	4	Kavya Rao	Priya Singh	2025-01-18	14:00:00.0000000	Completed
5	5	Rohan Verma	Anjali Gupta	2025-01-20	15:00:00.0000000	Completed
6	6	Priya Nair	Karan Mehta	2025-01-22	09:30:00.0000000	Completed
7	7	Aditya Gupta	Vikas Joshi	2025-01-25	10:30:00.0000000	Completed
8	8	Pallavi Joshi	Ritu Nair	2025-01-28	11:30:00.0000000	Completed
9	9	Nitin Singh	Suresh Iyer	2025-02-01	12:00:00.0000000	Completed
10	10	Meera Kapo...	Pooja Reddy	2025-02-03	13:00:00.0000000	Completed
11	11	Aakash Sha...	Amit Sharma	2025-02-05	09:00:00.0000000	Completed
12	12	Ritika Das	Neha Patel	2025-02-08	10:00:00.0000000	Completed
13	13	Vivek Reddy	Raj Verma	2025-02-10	11:00:00.0000000	Completed
14	14	Ananya Iyer	Priya Singh	2025-02-12	14:00:00.0000000	Completed
15	15	Deepak Mal...	Anjali Gupta	2025-02-15	15:00:00.0000000	Completed
16	16	Pooja Kulka...	Karan Mehta	2026-07-10	09:00:00.0000000	Scheduled
17	17	Sanjay Tiwari	Vikas Joshi	2026-07-12	10:00:00.0000000	Scheduled
18	18	Neelam Ban...	Ritu Nair	2026-07-15	11:00:00.0000000	Scheduled
19	19	Harsh Chopra	Suresh Iyer	2026-07-18	14:00:00.0000000	Scheduled
20	20	Divya Menon	Pooja Reddy	2026-07-20	15:00:00.0000000	Scheduled
21	21	Rahul Shah	Raj Verma	2026-07-22	09:00:00.0000000	Scheduled
22	22	Sneha Patel	Priya Singh	2026-07-25	10:00:00.0000000	Scheduled
23	23	Arjun Mehta	Anjali Gupta	2026-07-28	11:00:00.0000000	Scheduled
24	24	Kavya Rao	Karan Mehta	2026-08-01	12:00:00.0000000	Scheduled
25	25	Rohan Verma	Vikas Joshi	2026-08-03	13:00:00.0000000	Scheduled
26	26	Priya Nair	Ritu Nair	2025-03-10	09:00:00.0000000	Cancelled
27	27	Aditya Gupta	Suresh Iyer	2025-03-12	10:00:00.0000000	Cancelled
28	28	Pallavi Joshi	Pooja Reddy	2025-03-15	11:00:00.0000000	Cancelled
29	29	Nitin Singh	Amit Sharma	2025-03-18	14:00:00.0000000	Cancelled
30	30	Meera Kapo...	Neha Patel	2025-03-20	15:00:00.0000000	Cancelled

```sql
--2. LEFT JOIN
SELECT
    p.patient_id,
    p.first_name + ' ' + p.last_name AS patient_name,
    a.appt_id,
    a.appointment_date,
    a.status
FROM Patient p
LEFT JOIN Appointment a
    ON p.patient_id = a.patient_id;
```

	patient_id	patient_name	appt_id	appointment_date	status
1	1	Rahul Shah	1	2025-01-10	Completed
2	1	Rahul Shah	21	2026-07-22	Scheduled
3	2	Sneha Patel	2	2025-01-12	Completed
4	2	Sneha Patel	22	2026-07-25	Scheduled
5	3	Arjun Mehta	3	2025-01-15	Completed
6	3	Arjun Mehta	23	2026-07-28	Scheduled
7	4	Kavya Rao	4	2025-01-18	Completed
8	4	Kavya Rao	24	2026-08-01	Scheduled
9	5	Rohan Verma	5	2025-01-20	Completed
10	5	Rohan Verma	25	2026-08-03	Scheduled
11	6	Priya Nair	6	2025-01-22	Completed
12	6	Priya Nair	26	2025-03-10	Cancelled
13	7	Aditya Gupta	7	2025-01-25	Completed
14	7	Aditya Gupta	27	2025-03-12	Cancelled
15	8	Pallavi Joshi	8	2025-01-28	Completed
16	8	Pallavi Joshi	28	2025-03-15	Cancelled
17	9	Nitin Singh	9	2025-02-01	Completed
18	9	Nitin Singh	29	2025-03-18	Cancelled
19	10	Meera Kapoor	10	2025-02-03	Completed
20	10	Meera Kapoor	30	2025-03-20	Cancelled
21	11	Aakash Sha...	11	2025-02-05	Completed
22	12	Ritika Das	12	2025-02-08	Completed
23	13	Vivek Reddy	13	2025-02-10	Completed
24	14	Ananya Iyer	14	2025-02-12	Completed
25	15	Deepak Mal...	15	2025-02-15	Completed
26	16	Pooja Kulkarni	16	2026-07-10	Scheduled
27	17	Sanjay Tiwari	17	2026-07-12	Scheduled
28	18	Neelam Ban...	18	2026-07-15	Scheduled
29	19	Harsh Chopra	19	2026-07-18	Scheduled
30	20	Divya Menon	20	2026-07-20	Scheduled

```sql
--3. RIGHT JOIN
SELECT
    d.doctor_id,
    d.first_name + ' ' + d.last_name AS doctor_name,
    a.appt_id,
    a.appointment_date,
    a.status
FROM Appointment a
RIGHT JOIN Doctor d
    ON a.doctor_id = d.doctor_id;
```

	doctor_id	doctor_name	appt_id	appointment_date	status
1	1	Amit Sharma	1	2025-01-10	Completed
2	1	Amit Sharma	11	2025-02-05	Completed
3	1	Amit Sharma	29	2025-03-18	Cancelled
4	2	Neha Patel	2	2025-01-12	Completed
5	2	Neha Patel	12	2025-02-08	Completed
6	2	Neha Patel	30	2025-03-20	Cancelled
7	3	Raj Verma	3	2025-01-15	Completed
8	3	Raj Verma	13	2025-02-10	Completed
9	3	Raj Verma	21	2026-07-22	Scheduled
10	4	Priya Singh	4	2025-01-18	Completed
11	4	Priya Singh	14	2025-02-12	Completed
12	4	Priya Singh	22	2026-07-25	Scheduled
13	5	Anjali Gupta	5	2025-01-20	Completed
14	5	Anjali Gupta	15	2025-02-15	Completed
15	5	Anjali Gupta	23	2026-07-28	Scheduled
16	6	Karan Mehta	6	2025-01-22	Completed
17	6	Karan Mehta	16	2026-07-10	Scheduled
18	6	Karan Mehta	24	2026-08-01	Scheduled
19	7	Vikas Joshi	7	2025-01-25	Completed
20	7	Vikas Joshi	17	2026-07-12	Scheduled
21	7	Vikas Joshi	25	2026-08-03	Scheduled
22	8	Ritu Nair	8	2025-01-28	Completed
23	8	Ritu Nair	18	2026-07-15	Scheduled
24	8	Ritu Nair	26	2025-03-10	Cancelled
25	9	Suresh Iyer	9	2025-02-01	Completed
26	9	Suresh Iyer	19	2026-07-18	Scheduled
27	9	Suresh Iyer	27	2025-03-12	Cancelled
28	10	Pooja Reddy	10	2025-02-03	Completed
29	10	Pooja Reddy	20	2026-07-20	Scheduled
30	10	Pooja Reddy	28	2025-03-15	Cancelled

```sql
--4. MULTIPLE JOINS

SELECT
    p.first_name + ' ' + p.last_name AS patient_name,
    d.first_name + ' ' + d.last_name AS doctor_name,
    mr.diagnosis,
    mr.treatment
FROM Appointment a
INNER JOIN Patient p
    ON a.patient_id = p.patient_id
INNER JOIN Doctor d
    ON a.doctor_id = d.doctor_id
INNER JOIN Medical_Record mr
    ON p.patient_id = mr.patient_id
WHERE a.status = 'Completed';

--5. JOIN PATIENTS, APPOINTMENTS AND BILLING
SELECT
    p.patient_id,
    p.first_name + ' ' + p.last_name AS patient_name,
    COUNT(a.appt_id) AS total_appointments,
    SUM(b.amount) AS total_billing
FROM Patient p
LEFT JOIN Appointment a
    ON p.patient_id = a.patient_id
LEFT JOIN Billing b
    ON p.patient_id = b.patient_id
GROUP BY
    p.patient_id,
    p.first_name,
    P.last_name;

--6. DISPLAY DOCTOR NAME, DEPARTMENT NAME AND SPECIALISATION
SELECT
    d.first_name + ' ' + d.last_name AS doctor_name,
    dp.dept_name,
    d.specialization
FROM Doctor d
INNER JOIN Department dp
    ON d.dept_id = dp.dept_id;

--7. LIST ALL PRESCRIPTIONS WITH PATIENT NAME AND MEDICATION DETAILS
SELECT
    p.first_name + ' ' + p.last_name AS patient_name,
    pr.medication,
    pr.dosage
FROM Prescription pr
INNER JOIN Medical_Record mr
    ON pr.record_id = mr.record_id
INNER JOIN Patient p
    ON mr.patient_id = p.patient_id;
```

```sql
--8. SHOW APPOINTMENT DETAILS AND DEPARTMENT INFORMATION
SELECT
    a.appt_id,
    a.appointment_date,
    a.status,
    dp.dept_name,
    dp.location
FROM Appointment a
INNER JOIN Doctor d
    ON a.doctor_id = d.doctor_id
INNER JOIN Department dp
    ON d.dept_id = dp.dept_id;

--9. FIND PATIENTS WHO HAVE VISITED MULTIPLE DEPARTMENTS
SELECT
    p.patient_id,
    p.first_name + ' ' + p.last_name AS patient_name,
    COUNT(DISTINCT dp.dept_id) AS departments_visited
FROM Patient p
INNER JOIN Appointment a
    ON p.patient_id = a.patient_id
INNER JOIN Doctor d
    ON a.doctor_id = d.doctor_id
INNER JOIN Department dp
    ON d.dept_id = dp.dept_id
GROUP BY
    p.patient_id,
    p.first_name,
    p.last_name
HAVING COUNT(DISTINCT dp.dept_id) > 1;

--10. COMPREHENSIVE REPORT
SELECT
    p.first_name + ' ' + p.last_name AS patient_name,
    a.appointment_date,
    d.first_name + ' ' + d.last_name AS doctor_name,
    mr.diagnosis,
    b.amount AS bill_amount
FROM Patient p
INNER JOIN Appointment a
    ON p.patient_id = a.patient_id
INNER JOIN Doctor d
    ON a.doctor_id = d.doctor_id
LEFT JOIN Medical_Record mr
    ON p.patient_id = mr.patient_id
LEFT JOIN Billing b
    ON p.patient_id = b.patient_id
ORDER BY a.appointment_date;
```

TASK 10: SUBQUERIES

```sql
--1. Find patients who have appointments with doctors from 'Cardiology' department
SELECT *
FROM Patient
WHERE patient_id IN
(
   SELECT a.patient_id
   FROM Appointment a
   INNER JOIN Doctor d
      ON a.doctor_id = d.doctor_id
   INNER JOIN Department dp
      ON d.dept_id = dp.dept_id
   WHERE dp.dept_name = 'Cardiology'
);
```

	patient_id	first_name	last_name	date_of_birth	gender	phone	email	address	blood_group	insurance_provider	registration_date	is_active
1	1	Rahul	Shah	1990-05-10	Male	+91-9000000001	rahul.shah@email.com	Ahmedabad	A+	Star Health	2026-06-10	1
2	2	Sneha	Patel	1988-08-20	Female	+91-9000000002	sneha.patel@email.com	Mumbai	B+	ICICI Lombard	2026-06-10	1
3	9	Nitin	Singh	1987-06-12	Male	+91-9000000009	nitin.singh@email.com	Lucknow	A+	Star Health	2026-06-10	1
4	10	Meera	Kapoor	1993-01-30	Female	+91-9000000010	meera.kapoor@email.com	Indore	B+	HDFC Ergo	2026-06-10	1
5	11	Aakash	Sharma	1997-04-11	Male	+91-9000000011	aakash.sharma@email.com	Ahmedabad	O+	Care Health	2026-06-10	1
6	12	Ritika	Das	1996-10-09	Female	+91-9000000012	ritika.das@email.com	Kolkata	AB+	Star Health	2026-06-10	1

```sql
--2. List doctors who have more appointments than the average
SELECT
   doctor_id,
   COUNT(*) AS total_appointments
FROM Appointment
GROUP BY doctor_id
HAVING COUNT(*) >
(
   SELECT AVG(appointment_count)
   FROM
   (
      SELECT COUNT(*) AS appointment_count
      FROM Appointment
      GROUP BY doctor_id
   ) AS DoctorCounts
);

--3. Find the department with the most patients
SELECT TOP 1
   dp.dept_name,
   COUNT(DISTINCT a.patient_id) AS patient_count
FROM Department dp
INNER JOIN Doctor d
   ON dp.dept_id = d.dept_id
INNER JOIN Appointment a
   ON d.doctor_id = a.doctor_id
GROUP BY dp.dept_name
ORDER BY patient_count DESC;
```

	dept_name	patient_count
1	Emergency	6

```sql
--4. Show patients whose total billing is above averag
SELECT
   patient_id,
   SUM(amount) AS total_billing
FROM Billing
GROUP BY patient_id
HAVING SUM(amount) >
(
   SELECT AVG(total_bill)
   FROM
   (
      SELECT SUM(amount) AS total_bill
      FROM Billing
      GROUP BY patient_id
   ) AS BillingTotals
);
```

	patient_id	total_billing
1	2	12000.00
2	4	15000.00
3	7	18000.00
4	17	25000.00
5	19	50000.00

```sql
--5. List appointments scheduled on the same day as Rahul's appointment
SELECT *
FROM Appointment
WHERE appointment_date =
(
   SELECT TOP 1 a.appointment_date
   FROM Appointment a
   INNER JOIN Patient p
      ON a.patient_id = p.patient_id
   WHERE p.first_name = 'Rahul'
);

--6. Find doctors who have never had an appointment cancelled
SELECT *
FROM Doctor
WHERE doctor_id NOT IN
(
   SELECT DISTINCT doctor_id
   FROM Appointment
   WHERE status = 'Cancelled'
);

--7. Show patients who have been prescribed a specific medication
SELECT *
FROM Patient
WHERE patient_id IN
```

```
(
   SELECT mr.patient_id
   FROM Medical_Record mr
   INNER JOIN Prescription p
      ON mr.record_id = p.record_id
   WHERE p.medication = 'Aspirin'
);

--8. Find the most expensive medical procedure (highest billing amount)
SELECT *
FROM Billing
WHERE amount =
(
   SELECT MAX(amount)
   FROM Billing
);

--9. List departments where all doctors have at least one appointment

SELECT dept_name
FROM Department dp
WHERE NOT EXISTS
(
   SELECT *
   FROM Doctor d
   WHERE d.dept_id = dp.dept_id
   AND NOT EXISTS
   (
      SELECT *
      FROM Appointment a
      WHERE a.doctor_id = d.doctor_id
   )
);

--10. Find patients who visited the hospital more than 3 times
SELECT *
FROM Patient
WHERE patient_id IN
(
   SELECT patient_id
   FROM Appointment
   GROUP BY patient_id
   HAVING COUNT(*) > 3
);
```

TASK 11: CONDITIONAL LOGIC - CASE & IF

```sql
--1. Categorize Patients by Age
SELECT
   patient_id,
   first_name,
   last_name,
   DATEDIFF(YEAR, date_of_birth, GETDATE()) AS age,
   CASE
      WHEN DATEDIFF(YEAR, date_of_birth, GETDATE()) < 18
         THEN 'Child'
      WHEN DATEDIFF(YEAR, date_of_birth, GETDATE()) BETWEEN 18 AND 60
         THEN 'Adult'
      ELSE 'Senior'
   END AS age_category
FROM Patient;
```

	patient_id	first_name	last_name	age	age_category
1	1	Rahul	Shah	36	Adult
2	2	Sneha	Patel	38	Adult
3	3	Arjun	Mehta	31	Adult
4	4	Kavya	Rao	34	Adult
5	5	Rohan	Verma	41	Adult
6	6	Priya	Nair	28	Adult
7	7	Aditya	Gupta	35	Adult
8	8	Pallavi	Joshi	32	Adult
9	9	Nitin	Singh	39	Adult
10	10	Meera	Kapoor	33	Adult
11	11	Aakash	Sharma	29	Adult
12	12	Ritika	Das	30	Adult
13	13	Vivek	Reddy	37	Adult
14	14	Ananya	Iyer	35	Adult
15	15	Deepak	Malhotra	42	Adult
16	16	Pooja	Kulkarni	27	Adult
17	17	Sanjay	Tiwari	40	Adult
18	18	Neelam	Bansal	36	Adult
19	19	Harsh	Chopra	31	Adult
20	20	Divya	Menon	33	Adult

```sql
--2. Classify billing amounts
SELECT
   bill_id,
   amount,
   CASE
      WHEN amount < 5000
         THEN 'Small'
      WHEN amount BETWEEN 5000 AND 20000
         THEN 'Medium'
      ELSE 'Large'
   END AS billing_category
FROM Billing;
```

	bill_id	amount	billing_category
1	1	2500.00	Small
2	2	12000.00	Medium
3	3	1800.00	Small
4	4	15000.00	Medium
5	5	2200.00	Small
6	6	1200.00	Small
7	7	18000.00	Medium
8	8	3500.00	Small
9	9	5000.00	Medium
10	10	4200.00	Small
11	11	2500.00	Small
12	12	3000.00	Small
13	13	4500.00	Small
14	14	2000.00	Small
15	15	8000.00	Medium
16	16	1500.00	Small
17	17	25000.00	Large
18	18	900.00	Small
19	19	50000.00	Large
20	20	7000.00	Medium

```sql
--3. Mark appointments as recent or old
SELECT
   appt_id,
   appointment_date,
   CASE
      WHEN DATEDIFF(DAY, appointment_date, GETDATE()) <= 30
         THEN 'Recent'
      ELSE 'Old'
   END AS appointment_status
FROM Appointment;
```

	appt_id	appointment_date	appointment_status
1	1	2025-01-10	Old
2	2	2025-01-12	Old
3	3	2025-01-15	Old
4	4	2025-01-18	Old
5	5	2025-01-20	Old
6	6	2025-01-22	Old
7	7	2025-01-25	Old
8	8	2025-01-28	Old
9	9	2025-02-01	Old
10	10	2025-02-03	Old
11	11	2025-02-05	Old
12	12	2025-02-08	Old
13	13	2025-02-10	Old
14	14	2025-02-12	Old
15	15	2025-02-15	Old

16	16	2026-07-10	Recent
17	17	2026-07-12	Recent
18	18	2026-07-15	Recent
19	19	2026-07-18	Recent
20	20	2026-07-20	Recent
21	21	2026-07-22	Recent
22	22	2026-07-25	Recent
23	23	2026-07-28	Recent
24	24	2026-08-01	Recent
25	25	2026-08-03	Recent
26	26	2025-03-10	Old
27	27	2025-03-12	Old
28	28	2025-03-15	Old
29	29	2025-03-18	Old
30	30	2025-03-20	Old

```sql
--4. Assign priority to Appointments
SELECT
   a.appt_id,
   d.specialization,
   dp.dept_name,
   CASE
      WHEN dp.dept_name = 'Emergency'
         THEN 'Emergency'
      WHEN dp.dept_name IN ('Cardiology','Neurology')
         THEN 'Urgent'
      ELSE 'Regular'
   END AS priority_level
FROM Appointment a
INNER JOIN Doctor d
   ON a.doctor_id = d.doctor_id
INNER JOIN Department dp
   ON d.dept_id = dp.dept_id;

--5. Payment status description
SELECT
   bill_id,
   payment_status,
   CASE
      WHEN payment_status = 'Paid'
         THEN 'Settled'
      WHEN payment_status = 'Pending'
         THEN 'Due'
      ELSE 'Installment'
   END AS payment_description
FROM Billing;

--6. Categorise doctors by experience
SELECT
   doctor_id,
   first_name,
   last_name,
```

```sql
    CASE
        WHEN DATEDIFF(YEAR, '2015-01-01', GETDATE()) < 5
            THEN 'Junior'
        WHEN DATEDIFF(YEAR, '2015-01-01', GETDATE()) BETWEEN 5 AND 15
            THEN 'Mid-Level'
        ELSE 'Senior'
    END AS experience_level
FROM Doctor;

--7. Label patients as High risk
SELECT
    patient_id,
    COUNT(*) AS total_visits,
    CASE
        WHEN COUNT(*) > 5
            THEN 'High Risk'
        ELSE 'Normal'
    END AS risk_category
FROM Appointment
GROUP BY patient_id;

--8. Create appointment time slots
SELECT
    appt_id,
    appointment_time,
    CASE
        WHEN DATEPART(HOUR, appointment_time) BETWEEN 6 AND 11
            THEN 'Morning'
        WHEN DATEPART(HOUR, appointment_time) BETWEEN 12 AND 17
            THEN 'Afternoon'
        ELSE 'Evening'
    END AS time_slot
FROM Appointment;

--9. Assign Discount category based on loyalty
SELECT
    patient_id,
    registration_date,
    CASE
        WHEN DATEDIFF(YEAR, registration_date, GETDATE()) >= 5
            THEN 'Gold'
        WHEN DATEDIFF(YEAR, registration_date, GETDATE()) >= 2
            THEN 'Silver'
        ELSE 'Bronze'
    END AS loyalty_category
FROM Patient;

--10. Flag records requiring follow-up
SELECT
    record_id,
    diagnosis,
    CASE
        WHEN diagnosis LIKE '%Diabetes%'
            THEN 'Requires Follow-Up'
```

```
        WHEN diagnosis LIKE '%Hypertension%'
            THEN 'Requires Follow-Up'
        WHEN diagnosis LIKE '%Heart%'
            THEN 'Requires Follow-Up'
        ELSE 'Routine'
    END AS followup_status
FROM Medical_Record;
```

TASK 12: COMPLEX ANALYTICS QUERIES

```
--1. DEPARTMENT PERFORMANCE DASHBOARD

SELECT
    dp.dept_name,
    COUNT(DISTINCT a.patient_id) AS total_patients,
    COUNT(a.appt_id) AS total_appointments,
    SUM(b.amount) AS total_revenue,
    AVG(b.amount) AS avg_bill_amount
FROM Department dp
JOIN Doctor d
    ON dp.dept_id = d.dept_id
JOIN Appointment a
    ON d.doctor_id = a.doctor_id
LEFT JOIN Billing b
    ON a.patient_id = b.patient_id
GROUP BY dp.dept_name;
```

	dept_name	total_patients	total_appointments	total_revenue	avg_bill_amount
1	Cardiology	6	6	29200.00	4866.666666
2	Emergency	6	6	87700.00	14616.666666
3	Neurology	6	6	37800.00	6300.000000
4	Orthopedics	6	6	50800.00	8466.666666
5	Pediatrics	6	6	29700.00	4950.000000

```
--2. DOCTOR UTILIZATION REPORT

SELECT
    d.first_name + ' ' + d.last_name AS doctor_name,
    d.specialization,
    COUNT(a.appt_id) AS total_appointments,

    ROUND(
        100.0 *
        SUM(CASE WHEN a.status='Completed' THEN 1 ELSE 0 END)
        / COUNT(a.appt_id),
        2
    ) AS completion_rate,

    AVG(b.amount) AS avg_revenue_per_appointment

FROM Doctor d
LEFT JOIN Appointment a
    ON d.doctor_id = a.doctor_id
LEFT JOIN Billing b
    ON a.patient_id = b.patient_id

GROUP BY
    d.first_name,
    d.last_name,
    d.specialization;
```

	doctor_name	specialization	total_appointments	completion_rate	avg_revenue_per_appointment
1	Amit Sharma	Cardiologist	3	66.670000000000	3333.333333
2	Anjali Gupta	Pediatrician	3	66.670000000000	4000.000000
3	Karan Mehta	Pediatrician	3	33.330000000000	5900.000000
4	Neha Patel	Cardiologist	3	66.670000000000	6400.000000
5	Pooja Reddy	Emergency Specialist	3	33.330000000000	4900.000000
6	Priya Singh	Neurologist	3	66.670000000000	9666.666666
7	Raj Verma	Neurologist	3	66.670000000000	2933.333333
8	Ritu Nair	Orthopedic Surgeon	3	33.330000000000	1866.666666
9	Suresh Iyer	Emergency Specialist	3	33.330000000000	24333.333333
10	Vikas Joshi	Orthopedic Surgeon	3	33.330000000000	15066.666666

```sql
--3. PATIENT HEALTH SUMMARY
SELECT
    p.first_name + ' ' + p.last_name AS patient_name,
    COUNT(a.appt_id) AS total_visits,
    MAX(a.appointment_date) AS last_visit_date,
    SUM(b.amount) AS total_amount_spent,
    MAX(mr.diagnosis) AS recent_diagnosis

FROM Patient p

LEFT JOIN Appointment a
    ON p.patient_id = a.patient_id

LEFT JOIN Billing b
    ON p.patient_id = b.patient_id

LEFT JOIN Medical_Record mr
    ON p.patient_id = mr.patient_id

GROUP BY
    p.first_name,
    p.last_name;
```

	patient_name	total_visits	last_visit_date	total_amount_spent	recent_diagnosis
1	Neelam Bansal	1	2026-07-15	900.00	NULL
2	Harsh Chopra	1	2026-07-18	50000.00	NULL
3	Ritika Das	1	2025-02-08	3000.00	Anxiety
4	Aditya Gupta	2	2025-03-12	36000.00	Fractured Arm
5	Ananya Iyer	1	2025-02-12	2000.00	Allergy
6	Pallavi Joshi	2	2025-03-15	7000.00	Knee Pain
7	Meera Kapoor	2	2025-03-20	8400.00	Chest Infection
8	Pooja Kulkarni	1	2026-07-10	1500.00	NULL
9	Deepak Malhotra	1	2025-02-15	8000.00	High Cholesterol
10	Arjun Mehta	2	2026-07-28	3600.00	Migraine
11	Divya Menon	1	2026-07-20	7000.00	NULL
12	Priya Nair	2	2025-03-10	2400.00	Viral Fever
13	Sneha Patel	2	2026-07-25	24000.00	Heart Arrhythmia
14	Kavya Rao	2	2026-08-01	30000.00	Epilepsy
15	Vivek Reddy	1	2025-02-10	4500.00	Back Pain
16	Rahul Shah	2	2026-07-22	5000.00	Hypertension
17	Aakash Sharma	1	2025-02-05	2500.00	Diabetes
18	Nitin Singh	2	2025-03-18	10000.00	Food Poisoning
19	Sanjay Tiwari	1	2026-07-12	25000.00	NULL
20	Rohan Verma	2	2026-08-03	4400.00	Asthma

```sql
--4. MONTHLY REVENUE TREND
SELECT
    YEAR(bill_date) AS bill_year,
    MONTH(bill_date) AS bill_month,
    SUM(amount) AS monthly_revenue
FROM Billing
WHERE YEAR(bill_date)=YEAR(GETDATE())
GROUP BY
    YEAR(bill_date),
    MONTH(bill_date)
ORDER BY
    bill_month;

--5. PRESCRIPTION ANALYSIS
SELECT
    medication,
    COUNT(*) AS prescription_count
FROM Prescription
GROUP BY medication
ORDER BY prescription_count DESC;

--6. APPOINTMENT CANCELLATION ANALYSIS
SELECT
    d.first_name + ' ' + d.last_name AS doctor_name,

    COUNT(a.appt_id) AS total_appointments,

    SUM(
        CASE
            WHEN a.status='Cancelled'
```

```
            THEN 1
            ELSE 0
        END
    ) AS cancelled_appointments,

    ROUND(
        100.0 *
        SUM(CASE WHEN a.status='Cancelled' THEN 1 ELSE 0 END)
        / COUNT(*),
        2
    ) AS cancellation_rate

FROM Doctor d
JOIN Appointment a
    ON d.doctor_id = a.doctor_id

GROUP BY
    d.first_name,
    d.last_name;

--7.PATIENT SEGMENTATION
SELECT
    p.patient_id,

    COUNT(a.appt_id) AS visits,

    SUM(b.amount) AS spending,

    CASE
        WHEN COUNT(a.appt_id) >= 3
            AND SUM(b.amount) > 10000
        THEN 'Premium'

        WHEN COUNT(a.appt_id) >= 2
        THEN 'Regular'

        ELSE 'Occasional'
    END AS segment

FROM Patient p

LEFT JOIN Appointment a
    ON p.patient_id = a.patient_id

LEFT JOIN Billing b
    ON p.patient_id = b.patient_id

GROUP BY p.patient_id;

--8. DEPARTMENT EFFICIENCY
SELECT
    dp.dept_name,

    COUNT(a.appt_id) AS total_appointments,
```

```
    SUM(b.amount) AS department_revenue,

    AVG(b.amount) AS revenue_per_patient

FROM Department dp

JOIN Doctor d
    ON dp.dept_id=d.dept_id

JOIN Appointment a
    ON d.doctor_id=a.doctor_id

LEFT JOIN Billing b
    ON a.patient_id=b.patient_id

GROUP BY dp.dept_name;

--9. OUTSTANDING PAYMENTS REPORT
SELECT
    p.first_name,
    p.last_name,
    p.phone,
    b.amount,
    b.bill_date

FROM Billing b

JOIN Patient p
    ON b.patient_id=p.patient_id

WHERE b.payment_status='Pending'

ORDER BY b.amount DESC;

--10. COMPREHENSIVE HEALTH REPORT
SELECT
    p.first_name,
    p.last_name,
    p.gender,
    p.blood_group,
    mr.diagnosis,
    mr.treatment,
    pr.medication,
    b.amount,
    b.payment_status

FROM Patient p
LEFT JOIN Medical_Record mr
    ON p.patient_id=mr.patient_id
LEFT JOIN Prescription pr
    ON mr.record_id=pr.record_id
LEFT JOIN Billing b
    ON p.patient_id=b.patient_id;
```